

Payment API & integration guide

CLIENT AND DEVELOPER HANDBOOK | VERSION 1.0 | 19 SEPTEMBER 2026

A practical reference for connecting a shop to Payflow, creating PromptPay checkout links, checking payment status and completing client acceptance.

Current service

PromptPay QR collection with **manual administrator confirmation**. Funds go directly to the client's own bank-linked PromptPay destination. Payflow does not hold funds or initiate payouts.

The payment flow

Step	What happens
1. Shop	The shop server creates a payment and requests a checkout link using a private API key.
2. Customer	The buyer opens the client checkout, checks amount and recipient, then pays with a bank app.
3. Bank receipt	The client receives the funds directly. An authorized Payflow admin checks the actual bank record.
4. Confirmation	The admin confirms receipt. The API reports succeeded; the shop validates the result and fulfills once.

Inside this guide

Page	Reference
02	Connection, authentication and endpoint directory
03	Create a payment: request fields and response
04	Create a checkout link and handle asynchronous processing
05	Read payment status and validate fulfillment
06	Read operations, retry safely and avoid duplicates
07	Errors and troubleshooting
08	Checkout domain setup and security
09	Endpoint test plan and real payment acceptance
10	Scope boundaries, launch checklist and source references

Examples use placeholders and fictional order IDs. No credential, real customer record or live checkout token is included. All request paths are relative to the API base URL unless stated otherwise.

Connect to the API

Use a server-side HTTP client, Postman or your shop backend. The API origin is not a browser dashboard.

Surface	Address and purpose
API base URL	https://api.gyroinst.com/gateway/v1 Authenticated shop requests.
Operator dashboard	https://payments.gyroinst.com/ Private operator access; root redirects to login when signed out.
Client checkout	https://<verified-client-host>/pay/<token> Use the exact URL returned by the checkout endpoint.

Authentication and headers

```
Authorization: Bearer <PRIVATE_LIVE_API_KEY>
Accept: application/json
Content-Type: application/json
Idempotency-Key: <unique-key-for-this-POST>
```

Send the bearer key on every API request. Send JSON and an Idempotency-Key on every POST. GET requests do not require an idempotency key. Keys are scoped to one merchant and environment; the key determines which records are accessible.

Keep the key on your server. The existing **pk_** prefix does not make it public. Do not put it in browser code, URLs, screenshots, logs or source control. Revoke and replace a compromised key.

Live-only deployment

The current deployment rejects test keys with 403 / `test_mode_disabled`. A valid live key needs `payments.write` to create records and `payments.read` to retrieve them. A generated live QR is payable even during integration testing.

Supported endpoint directory

Method and path	Scope	Success
POST /payments	payments.write	201; payment object
POST /payments/{payment}/checkout-sessions	payments.write	201 with checkout, or 202 with operation
GET /payments/{payment}	payments.read	200; payment object
GET /operations/{operation}	payments.read	200; operation and optional checkout

Path values {payment} and {operation} are the IDs returned by the API. These endpoints are not prefixed with /api. An active, verified client checkout domain is required for **both** payment creation and checkout creation.

Create a payment

POST /gateway/v1/payments | Scope: payments.write | Success: 201

```
POST https://api.gyroinst.com/gateway/v1/payments
Authorization: Bearer <PRIVATE_LIVE_API_KEY>
Content-Type: application/json
Accept: application/json
Idempotency-Key: api-check-001-payment

{
  "order_ref": "API-CHECK-001",
  "amount_minor": 100,
  "currency": "THB",
  "metadata": {"order_id": "API-CHECK-001"}
}
```

Field	Requirement
order_ref	Required string, up to 191 characters. Stable shop order identifier; reuse on retries.
amount_minor	Required positive integer in satang. 100 = THB 1.00. Maximum 99,999,999; a lower configured live limit may apply.
currency	Required. Use THB for the current PromptPay integration.
payment_purpose	Optional string, up to 64 characters. Defaults to order; keep stable on retries.
metadata	Optional object. Only order_id, invoice_id and reference; each value is a string of up to 191 characters or null.

Optional customer fields

customer_ref (191 characters) is needed to persist a customer profile. Optional fields: customer_name, company_name and customer_email (191 each; email must be valid); customer_phone and customer_type (64 each). identification_type (64) and identification_number (191) are also accepted customer declarations, not bank-verified identity. Omit sensitive identity data unless separately agreed.

return_origin is optional for PromptPay; omit it. This checkout does not automatically redirect to the shop. Compute the order amount on your server. The receiving account comes from operator configuration and cannot be changed by this API.

201 response - selected fields

```
{"payment": {
  "id": "<payment-uuid>", "order_ref": "API-CHECK-001",
  "status": "created", "currency": "THB",
  "requested": "1.00", "captured": "0.00", "is_demo": false
}}
```

Save payment.id with the shop order. Request amounts are integer satang; response amounts are decimal strings in baht. A created payment is not a receipt.

Create the checkout link

POST /gateway/v1/payments/{payment}/checkout-sessions | Scope: payments.write

```
POST /gateway/v1/payments/<payment-uuid>/checkout-sessions
Authorization: Bearer <PRIVATE_LIVE_API_KEY>
Accept: application/json
Content-Type: application/json
Idempotency-Key: api-check-001-checkout

{}
```

Use a separate idempotency key from payment creation. The empty JSON object is sufficient for PromptPay. The payment must belong to the same merchant and environment as the key.

201 response - selected fields

```
{
  "operation": {
    "id": "<operation-uuid>",
    "action": "checkout_session.create",
    "state": "succeeded"
  },
  "checkout": {
    "session_ref": "pp_<link-uuid>",
    "redirect_url": "https://<client-host>/pay/<64-character-token>",
    "expires_at": "<ISO-8601-timestamp>"
  }
}
```

Checkout success is not payment success

operation.state = succeeded means the checkout link was created. Only the authenticated payment response can establish the final paid state after manual bank review.

If the response is 202

```
{
  "operation": {"id": "<operation-uuid>", "state": "<current-state>"},
  "poll": "/gateway/v1/operations/<operation-uuid>"
}
```

Save operation.id and poll the supplied path against <https://api.gyroinst.com> every 3-5 seconds, then back off. Do not append that path to the /gateway/v1 base twice. When a usable checkout is returned, open its exact redirect_url.

Link behavior

One enabled client domain is selected and stored with the link. Retries keep the same host and token; page reloads do not rotate domains. Links expire after 24 hours. Keep URLs private, do not rewrite them, and do not use the dashboard as a checkout fallback.

Read status and fulfill once

GET /gateway/v1/payments/{payment} | Scope: payments.read | Success: 200

```
GET /gateway/v1/payments/<payment-uuid>
Authorization: Bearer <PRIVATE_LIVE_API_KEY>
Accept: application/json
```

After actual bank receipt and manual confirmation

```
{
  "payment": {
    "id": "<payment-uuid>",
    "order_ref": "API-CHECK-001",
    "status": "succeeded",
    "currency": "THB",
    "requested": "1.00",
    "captured": "1.00",
    "is_demo": false,
    "verification_source": "manual_bank_review"
  }
}
```

Illustrative response subset. The admin verifies the client's actual receiving-bank record and confirms with permission, fresh MFA, password and a unique bank reference. There is no client API endpoint to mark a payment paid.

Status	Shop behavior
created / awaiting_payment	Wait. Creating or viewing the QR does not confirm receipt.
processing / authorized	Wait. Do not interpret these general model states as paid.
succeeded	Validate identity, amount, currency and verification source; fulfill once.
failed / canceled / expired	Do not fulfill. Review the order with the operator.
unknown	Ask the operator to investigate before taking further action.

Fulfillment acceptance rule

Require all of: matching payment.id and order_ref; status = succeeded; currency = THB; requested and captured equal the order total; is_demo = false; verification_source = manual_bank_review. Compare money using integer satang or a decimal type, never binary floating-point arithmetic.

Use a durable fulfillment marker or unique order constraint so repeated polls cannot deliver twice. Poll active payments about every 5-10 seconds, then back off. Treat the public checkout status as display-only.

Other response fields include payment_purpose, provider_status, refunded, refundable, capture_state, refund_state, settlement_state, version, observed_at, created_at and an optional customer object. Ledger fields do not mean bank refunds or payouts are available.

Operations, retries and duplicates

GET /gateway/v1/operations/{operation} | Scope: payments.read | Success: 200

```
GET /gateway/v1/operations/<operation-uuid>
Authorization: Bearer <PRIVATE_LIVE_API_KEY>
Accept: application/json
```

The response contains operation and checkout. checkout may be null until a usable session exists, or if the link is no longer usable. A completed operation does not guarantee the checkout is still available.

Operation field	Meaning
id / payment_id / action	Operation identity, associated payment and action such as checkout_session.create.
state / terminal / unresolved	Processing state and flags describing completion or unresolved outcome.
requires_approval	Whether an operator approval is required for the operation.
error_code / error_message	Reason to investigate when the operation cannot complete. Values may be null.
attempts / created_at / completed_at	Attempt count and timestamps. completed_at may be null.

Stop and investigate

For failed, rejected, outcome_unknown or needs_investigation, contact the operator. A terminal operation without a usable checkout also needs review. Do not create a new order to bypass an unresolved result.

Safe retry rules

Situation	Required behavior
Timeout or lost response	Repeat the same method, URL, idempotency key and JSON. Keep the same order_ref.
Checkout returned 202	Poll the operation. Replaying the POST may repeat the original 202 response rather than the latest result.
Same key, changed request	Expect 409 idempotency_key_reuse. Do not reuse keys for different actions or payloads.
Same business order, new key	A stable order_ref plus payment_purpose protects the same obligation. Conflicting details are rejected; do not use new keys to force duplicates.
429 rate limit	Respect Retry-After, reduce concurrency and retry with increasing delay plus jitter.

The authenticated gateway limit is 120 requests per minute per key. Perimeter protection may impose additional limits. Successful request replays may include Idempotency-Replayed: true. Retain order and fulfillment records independently of HTTP retries.

Errors and troubleshooting

Use the HTTP status together with the JSON response. Validation and framework errors can have a different envelope.

```
{
  "error": {
    "code": "checkout_domain_unavailable",
    "message": "<operator setup explanation>"
  }
}
```

HTTP	Meaning	Next action
400	Missing Idempotency-Key	Add a stable key for this POST.
401	Missing, invalid, expired or revoked credentials	Check the Bearer header, key status and merchant availability.
403	Insufficient scope or wrong environment	Check payments.read/write and use a live key. Test keys are disabled here.
404	Wrong route or inaccessible record	Check host/path, returned ID, merchant and environment.
405	Wrong HTTP method	Payment creation uses POST; typing the URL in a browser sends GET.
409	Setup or request conflict	Read error.code. Activate the checkout domain, configure the provider account, or resolve order/idempotency conflicts.
422	Invalid request fields	Correct the fields named in errors; verify integer satang and THB.
429	Too many requests	Honor Retry-After and back off.
500 / 503	Server failure or disabled/unavailable service	Retry conservatively with the same key; contact the operator if persistent.

Common setup checks

checkout_domain_unavailable: the merchant needs an active, verified checkout domain before either creation endpoint can run. Registering a hostname alone does not establish DNS, TLS or routing.

no_provider_account: the operator must configure an enabled PromptPay provider account for the merchant and live environment. **test_mode_disabled:** a test key was used on the live-only deployment.

Browser URLs and perimeter responses

<https://api.gyroinst.com/> and its /login path return 404 intentionally. Use /gateway/v1 endpoints with the correct method. The dashboard root should redirect to /login. A plain HTML 403 may come from the web server or firewall rather than API JSON; record the time, host, method and sanitized path for diagnosis.

Share only sanitized IDs, status codes and error messages with support. Do not include API keys, full checkout tokens, bank credentials or unnecessary customer data.

Checkout domains and security

The client checkout domain is separate from the operator dashboard and the authenticated API.

Provide the exact client-owned hostname. The operator registers it, verifies control and secure routing, then enables it. Hostnames are assigned to one merchant. The API and admin hostnames cannot be reused as client checkout domains.

Client checkout path	Method	Purpose
/pay/{token}	GET / HEAD	Payment page; token is 64 lowercase hexadecimal characters.
/pay/{token}/qr	GET / HEAD	QR image.
/pay/{token}/status	GET / HEAD	Buyer display status, not the fulfillment authority.
/assets/promptpay.css	GET / HEAD	Checkout stylesheet.
/assets/payment-status.js	GET / HEAD	Browser status polling.
/checkout-health	GET / HEAD	Routing probe on a registered hostname; expected text: checkout-ready.

Connection requirements

Use valid HTTPS with a certificate for the client hostname. Coordinate DNS and gateway host admission with the operator. If using a reverse proxy, forward only the approved checkout paths, preserve the original Host and HTTPS scheme, and verify TLS to the origin. DNS changes alone are insufficient.

Do not iframe the checkout, rewrite its link to another host or cache its HTML, QR or status. Preserve no-store and no-referrer protections. Exclude token paths from analytics and logs. Do not expose dashboard routes or cookies through the checkout domain.

Security responsibilities

The platform uses HTTPS/TLS, web request filtering and rate limits. API credentials are scoped; records are isolated by merchant and environment. Administrators use protected login and MFA for receipt confirmation. Keep shop-side credentials private and use normal certificate validation.

Money and credential boundary

Customers authorize transfers in their bank app. Payflow does not collect bank passwords, PINs, OTPs, card numbers or CVV, and does not hold client funds. A screenshot, scanned QR or bank notification alone is not confirmation of receipt.

Expiry and revocation restrict the application link, not a saved bank QR. A copied QR can still receive a transfer. Equal-amount payments to the same recipient may have identical QR data; reconcile each actual bank receipt carefully.

Test before accepting orders

Begin with API and link checks without transferring money. Complete a separately authorized small real payment afterward.

Stage	Check	Expected outcome
1. Access	POST /payments without a bearer key	401. No payment is created.
2. Setup	Confirm live key/scopes, client recipient, provider mapping and active checkout domain	All prerequisites accepted for the intended merchant.
3. Create	Use a new order_ref and amount_minor = 100	201; requested = "1.00", status = created and is_demo = false.
4. Replay	Repeat the identical POST with the same key	Same payment ID; no duplicate order.
5. Checkout	POST checkout-sessions with a separate key; poll if 202	Usable URL on the correct verified client domain.
6. Inspect	Open the link without paying	Correct recipient and amount. Payment remains unpaid.
7. Receipt	Authorize one small transfer into the client account; admin checks the actual bank entry and confirms	Authenticated API reports succeeded and manual_bank_review.
8. Fulfillment	Verify order/amount/currency, fulfill, then poll again	Exactly one fulfillment; repeated success does not duplicate delivery.

Use the intended receiving account

The platform owner's reported 1 baht test proves only that earlier test. Client acceptance must use the client's own confirmed receiving destination. Creating a live checkout does not transfer money, but its QR is payable.

Postman setup

Import the companion docs/openapi.json file from the project. Set the server to <https://api.gyroinst.com> and use a private local secret variable for the bearer key. Add separate idempotency keys to the two POST requests. Save payment.id and operation.id for subsequent calls. Do not sync keys or publish a collection containing credentials.

Additional checks before launch

Verify wrong-scope and revoked keys fail; one merchant cannot read another's payment; altered retries are rejected; disabled domains cannot issue new links; and expired/revoked checkout behavior is handled. Use designated test credentials and records when changing access or domain state.

This guide does not perform a transfer or certify a client. Record sanitized request results, bank review, admin confirmation and one-time shop fulfillment. Do not send a second payment just because the page has not updated.

Service boundaries and go-live

Platform implementation is complete for manual PromptPay collection. Client-specific acceptance remains a separate requirement.

Routes outside the supported PromptPay service

Registered route	Current service boundary
POST /payments/{payment}/refunds	Generic route exists and requires refunds.write, but PromptPay cannot execute a bank refund. Do not integrate it as a supported refund service.
POST /payments/{payment}/captures	Capability-gated; unavailable for this flow. Capture does not replace manual bank receipt review.
POST /payments/{payment}/cancel	Capability-gated; unavailable for this flow. An application restriction cannot reverse a bank transfer.

The four supported endpoints are listed on page 2. Stripe is retired. The Android notification relay is disabled and outside this integration. There is no official bank webhook or automatic bank receipt detection. Refunds/returns are arranged through the bank; outbound payouts are outside scope.

Use authenticated polling for shop status. Optional merchant callbacks are not part of the initial integration; dedicated signing secrets must be implemented and accepted before external use. Never distribute the platform application key.

Client launch checklist

1. Client-owned receiving details confirmed and saved under the correct merchant.
2. Private live API key and shop integration accepted.
3. Verified active checkout domain with working DNS, TLS and routing.
4. Small real payment, bank review, manual confirmation and one-time fulfillment passed.
5. Payment-review coverage, unmatched-payment handling, bank refunds, monitoring and recovery agreed.

Document basis and operational snapshot

Prepared 19 September 2026 from the current client API guide, OpenAPI 3.1 specification (API version 1.0.0), domain contract, routes, controllers and middleware. Response examples are selected fields, not complete schemas. The machine-readable contract is **docs/openapi.json**.

Dated verification: both serving targets were healthy, using the deployed b351 release in PromptPay live-only mode with relay disabled. The dashboard homepage repair is applied; replacement-server configuration uses launch-template version 9. Final3 remains undeployed. This is a dated operational snapshot, not a guarantee of future availability.

Maintained project reference	Use
docs/CLIENT-API.md and docs/openapi.json	Integration contract and importable API schema.
docs/CHECKOUT-DOMAINS.md	Client domain connection and activation.
PROJECT_SCOPE.md and PROMPTPAY.md	Product boundaries and manual confirmation.
docs/PRODUCTION-READINESS.md and RUNBOOK.md	Dated evidence, acceptance and operations.